Case-Based Reasoning in Design

Mary Lou Maher and Andrés Gómez de Silva Garza, University of Sydney

CASE-BASED REASONING IS A GENeral paradigm for problem solving based on the recall and reuse of specific experiences. AI practitioners have applied and extended this paradigm for numerous classes of design problems, indicating that it is an important aspect of design support. This survey considers the range and types of design applications that have implemented case-based reasoning and shows how these applications have resolved theoretical issues through their implementations.

THIS SURVEY PRESENTS A NUMBER OF ISSUES RAISED BY THE APPLICATION OF CASE-BASED REASONING FOR DESIGN. THE AUTHORS COMMENT ON THE TWO EXTREMES OF CASE-BASED DESIGN: DESIGN ASSISTANCE AND DESIGN AUTOMATION, AND HOW THEY SHARE A NUMBER OF DIFFICULT ISSUES RELATED TO THEIR IMPLEMENTATION.

Learning to design

CBR supports design by reminding designers of previous experiences that can help with new situations. As designers, we learn to design by experiencing design situations. This is also how we teach students to be designers. For example, engineers learn to have analytical capabilities in school and then learn to design in professional practice; architects learn through a range of design experiences in the studio. As designers, we learn to analyze through the use of formal methods, but creating a new design requires previous experience, or at least, exposure to another's design experiences. Current formal methods for generating designs are useful for programming computers to design, but not necessarily as models of human thinking. In CBR, there are similarities between the formal method implemented in a computer program and the informal observation that human designers rely heavily on specific design experiences.

CBR's problem-solving approach relies on analogy, where problems or experiences outside the one we are currently addressing might provide some insight or assistance. Through analogy, we might be reminded of a window design when designing a door to a balcony with a view. Analogy lets us recognize something that we have not encountered before, by associating it with something we have.

Considering analogy from the perspective of memory and reminding has led to the concept of memory organization as a guideline for computer representations.[1] The study of memory organization, and its use as a basis for solving new problems and generating explanations, has led to models for representing experience in computers.[2] This AI research area has found use in real-world situations.[3,4]

CBR formalizes a computational model of problem solving based on memory organization and reminding. In addition to addressing memory organization, CBR provides a process model with specific stages. CBR lets us implement computer support for designers by applying memory-organization research to define a case memory of design experience and by applying recall and adaptation processes to reuse the experience.

Both CBR and expert systems rely on the explicit symbolic representation of experience-based knowledge to solve a new problem. However, expert systems use past expe-

0885-9000/97/$10.00 © 1997 IEEE

rience stored in a knowledge base of generalized heuristics to assist in solving a new problem.[5] They can store the generalized heuristics as rules of thumb or as logical inferences. CBR uses an abstraction of specific problem-solving experiences to learn to solve a new problem. The representation of specific experiences usually includes the justification of the solution or the requirements of the problem as well as its solution.

Two major considerations in a CBR approach to design are

- representations of design cases and
- process models for recalling and adapting design cases.

A high-level view of CBR illustrates the role of design cases in the CBR process (see Figure 1). Representing design cases requires an abstraction of the experience into a symbolic form that the reasoner (either computer or human) can manipulate. As this survey shows, practitioners have developed various abstractions based on models, methods, and philosophical approaches to understanding design, designers, and design experiences.

Design-case recall involves finding a *relevant* design experience; it decomposes into the subtasks of *indexing*, *retrieval*, and *selection* (see Figure 2a). Indexing identifies the features a previous design should have to be relevant to the new design problem. Retrieval identifies which cases have all or a subset of those features, and selection evaluates the retrieved cases so they can be ranked.

Design-case adaptation recognizes the differences between the selected design case and the new design problem, and changes the design case so that it solves the new design problem. This process decomposes into three steps: *propose*, *evaluate*, and *modify* (see Figure 2b). Before starting on a modify-evaluate cycle, the process involves proposing the selected design case as a solution to the new design problem. Evaluation checks the feasibility of the new design solution. Modification changes parts of the new design solution, thus transforming it into a different proposed solution that must then be evaluated.

Applications of CBR to design

Several recent publications have addressed specific considerations of the application of CBR to design.[6–8] These applications have ranged from largely informal domains, such as meal planning, to analytical domains, such as the design of mechanical devices. Each design application brings a new set of considerations to the use of CBR in design. The application of CBR to design has produced a number of implementations that resolve theoretical issues. Each implementation serves as an example for those who intend to use case-based design as a computer support system for designers. Table 1 lists a selection of implementations that serve as the basis for our discussion. The domains shown reflect the range of issues introduced by the need to develop practical applications.

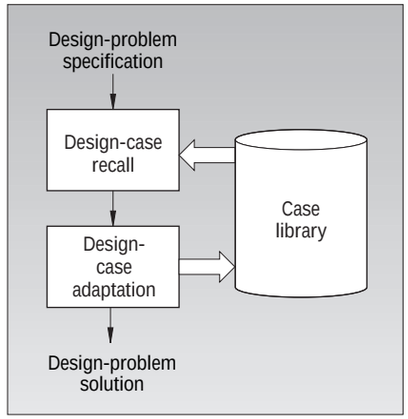


Figure 1. The overall process model of case-based design.

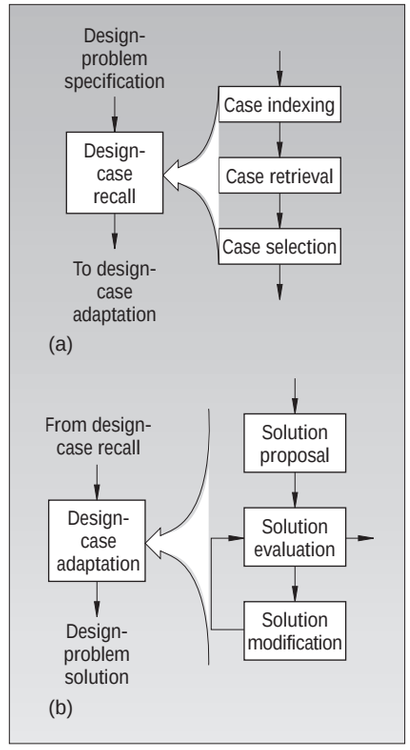


Figure 2. Generic processes: (a) case recall; and (b) case adaptation.

We consider these recurring themes in the implementation of case-based design:

- the need to represent and manage *complex design cases*,
- the need to augment a set of specific experiences with *generalized design knowledge*, and
- the need to formalize a typically *informal body of knowledge* or experiences.

Complex design cases. Design in any domain usually involves the development and understanding of complex systems. The complex representations needed to adequately capture a design case have introduced challenges to the implementation of case-based design. The CBR paradigm assumes that there is a concept of "a case"; in most design domains, however, a design is not simply "a case" but a complex set of experiences and decisions resulting in a complex system. Research has addressed this complexity issue in three ways:

- representing a design case as a *partonomic hierarchy*,
- representing *different views* of the same design case, and
- using *multimedia* representations for different types of information.

These three approaches are not exclusive; one implementation can use more than one approach.

A partonomic hierarchy of a design case is a decomposition of the design experience into a hierarchy of parts or subcases. This decomposition allows reasoning to focus only on the relevant parts of a design. Processing only some of the knowledge associated with a case lets reasoning become more efficient. The development of a hierarchically structured case base requires defining a typical decomposition of a design experience. System designers might represent this decomposition explicitly through design models, or use it implicitly for case acquisition.

Using a partonomic hierarchy is the most common way to address design complexity. System developers used this approach, with slight variations, in Casecad, Cadsyn, Gencad, Panda, Asp-II, and Benton. For example, Casecad and Cadsyn represent subcases with the *Function Behavior Structure* (FBS) framework. This framework classifies the variables used to describe design cases into three semantically different categories. *Function variables* describe aspects of a design related to its purposes and the designer's

Table 1. Applications of CBR to design.

System name	Design domain	Reference
Archie, Archie-II	Architecture	E. Domeshek and J. Kolodner, "The Designer's Muse," in *Issues and Applications of Case-Based Reasoning in Design*, M.L. Maher and P. Pu, eds., Lawrence Erlbaum Associates, Hillsdale, N.J., 1997, pp. 11–38.
AskJef	Software	J. Barber et al., "AskJef: Integration of Case-Based and Multimedia Technologies for Interface Design Support," in *Artificial Intelligence in Design*, Kluwer Academic Publishers, Dordrecht, The Netherlands, pp. 457–475.
Asp-II, Benton	Software	C. Tsatsoulis and P. Alexander, "Integrating Cases, Sub-Cases, and Generic Prototypes for Design," in *Issues and Applications of Case-Based Reasoning in Design*, pp. 261–300.
Cadet	Mechanical devices	S. Narashiman, K. Sycara, and D. Navin-Chandra, "Representation and Synthesis of Non-Monotonic Mechanical Devices," *Issues and Applications of Case-Based Reasoning in Design*, pp. 187–220.
Cadre, Faming	Architecture	B. Faltings, "Case Reuse by Model-Based Interpretation," in *Issues and Applications of Case-Based Reasoning in Design*, pp. 39–60.
Cadsyn, Casecad	Structural systems	M.L. Maher, B. Balachandran, and D.M. Zhang, *Case-Based Reasoning in Design*, Lawrence Erlbaum, 1995.
Fabel	Architecture	A. Voss, "Case Design Specialists in Fabel," in *Issues and Applications of Case-Based Reasoning in Design*, pp. 301–338.
Gencad	Structural systems	A. Gómez de Silva Garza and M.L. Maher, "The Adaptation of Structural System Designs Using Genetic Algorithms," *Proc. Int'l Conf. Information Technology in Civil and Structural Engineering Design*, Elsevier Science, New York, 1996, pp. 189–196.
Janus	Architecture	G. Fisher, R. McCall, and A. Morch, "Design Environments for Constructive and Argumentative Design," *Proc. Human Factors in Computing Systems Conf.*, ACM Press, New York, 1989, pp. 269–275.
Julia	Meal planning	T.R. Hinrichs, "Towards an Architecture for Open World Problem Solving," *Proc. CBR Workshop*, Morgan Kaufmann, San Francisco, 1988, pp. 182–189.
Kritik, Kritik II	Physical devices	A. Goel, S. Bhatta, and E. Stroulia, "KRITIK: An Early Case-Based Design System," *Issues and Applications of Case-Based Reasoning in Design*, pp. 87–132.
Panda	Fire engines	C. Tsatsoulis and P. Alexander, "Integrating Cases, Sub-Cases, and Generic Prototypes for Design," in *Issues and Applications of Case-Based Reasoning in Design*, pp. 261–300.
Seed	Architecture	U. Flemming et al., "Case-Based Design in a Software Environment That Supports the Early Phases," in *Issues and Applications of Case-Based Reasoning in Design*, pp. 61–86.

intentions. *Behavior variables* are features of a design description that concern the design's interaction with and response to its environment. *Structure variables* describe the design's physical, geometric, and visual properties. Gencad adds the notion of *context variables*, which describe features of the design's environment that might influence design decisions.

Fabel addresses the complexity of the building-design domain by using different views or aspects of a specific design. The Fabel system can represent a single design problem as several design cases, where each case is a different perspective on the design. This approach involves using different views of the design, such as *gestalts* (simplified holistic abstractions of a complex geometry), *attribute vectors* (lists of values of predetermined features), and *images* (digital images of the design solution). This approach distinguishes itself from the partonomic hierarchy by treating each view as a case and not necessarily relating all views of a case to make up a "whole" case description.

The multimedia approach treats complexity by organizing the information about a design around different storage formats, then cross-referencing the different representations. For example, in Archie-II, a design can consist of a building's photograph, a drawing of its architectural plan, and textual annotations about different aspects of the building's design. AskJef contains symbolic, graphical, and auditory representations for cases. AskJef also decomposes cases temporally, so that each case is actually a collection of different versions of a design that were generated iteratively during the design phase (only the last one of which was deemed to be a good solution).

In summary, designers address complexity by breaking a case into smaller, less complex abstractions. Table 2 summarizes the approaches used for breaking down a building design case, with example systems cited (the list of example systems is not exhaustive—other systems mentioned in this survey are not cited in Table 2, and many systems not mentioned here might also follow each approach).

Generalized design knowledge. The application of CBR to design domains that involve mechanical and other physical devices has extended the notion of design cases to include a representation of general knowledge in the form of causal or heuristic knowledge. In such design domains, the representation and reuse of design cases is impossible without some generalized knowledge. Using the terminology of the authors of case-based design systems, some methods for representing generalized design knowledge include

- causal models,
- state interactions,
- heuristic models,
- heuristic rules, and
- geometric constraints.

Kritik's developers directly addressed the representation of causal behavior through the use of *structure-behavior-function* models. (The SBF formalism uses the same words as the FBS formalism, but with fundamentally different meanings.) Kritik represents behaviors as sets of device states and transitions linking the states; these behaviors explain how the physical device's structural elements

Table 2. Approaches to complex design-case representation.

Approach	System	Example
Partonomic hierarchies	Casecad	Building as lateral and gravity systems.
Views	Fabel	Building as layout, topology, or gestalt.
Multimedia	Archie-II	Building as photographs, drawings, or stories.

Table 3. Approaches to representing generalized knowledge.

Approach	System	Example
Causal models	Kritik	Structure, behavior, and function interactions
State interactions	Cadet	Interactive state diagrams
Heuristic models	Casecad	Function, behavior, and structure attribute value ranges
Heuristic rules	Panda	Transformation rules
Geometric constraints	Cadre	Parametric constraints

achieve its functions. Causal knowledge in the form of SBF models allows Kritik to diagnose and repair faulty or malfunctioning devices. Like Kritik, Cadet uses *interactive state diagrams* to represent device behavior, using graphs with nodes and transitions between the nodes. A device's behavior in Cadet, as in Kritik, explains how it achieves its functions.

In contrast to the models used in Kritik and Cadet, Casecad's FBS models identify a range of allowable values for each variable. These ranges define different models that cover sets of cases belonging to different classes, rather than a specific case, and are more like heuristic knowledge than causal knowledge.

Many systems represent generalized knowledge as constraints that define dependencies among variables. Cadsyn contains generalized knowledge in the form of constraints on the values of variables and their relationships that good designs must satisfy. By definition, these constraints apply to all design cases. Faming and Cadre store generalized knowledge as case-specific constraints.

Systems also use rules to capture generalized knowledge. Many implementations distinguish rules from constraints by applying rules to actively *change* a proposed design solution and constraints to evaluate the feasibility of a proposed design solution, corresponding to the modify and evaluate stages shown in Figure 2b. Panda uses rules to transform a retrieved design case to be feasible for a new design problem. Janus represents generalized knowledge in the form of rules that can help critique potential designs. Archie provides domain models as qualitative rules about how changes to one variable can affect other variables.

Table 3 summarizes the different approaches to representing generalized knowledge in case-based design systems. In some systems, the generalized knowledge dominates the representation of design cases; for example, Kritik's use of causal models provides the focus for case retrieval and adaptation. Other systems only need the generalized knowledge during case adaptation and represent it independently of a design case—for example, Panda's use of transformation rules.

Informal nature of domains. The lack of formal knowledge in design affects both the ability to define a formal and consistent representation of design cases and the role of adaptation as either a human-centered activity or an automated process. We can address the development of CBR for design domains having little formal or theoretical knowledge either by formalizing knowledge that previously was not formalized, or by identifying a representation of the design cases to support *human* rather than automated reasoning.

Cadre and Archie are CBR approaches to architectural design, a design domain characterized by a lack of formal, analytical knowledge. The Cadre project focuses on areas of architectural design that can be formalized, such as the geometric constraints on floor plans. Cadre represents design cases as existing floor plans and uses numerical constraints on the dimensions of the required floor plan to automatically modify and evaluate the adaptation and combination of floor plans.

The Archie project looks at how issues of importance during conceptual design can be articulated and presented to designers as multimedia. In this implementation, the design knowledge is not formalized for automated reasoning, but articulated and expressed in a form that aids people in reusing architectural principles. The use of multimedia and hypertext environments provides a rich representation and flexible use of the informal knowledge.

Julia develops a formal representation of a previously informal task, the task of meal planning, allowing it to be automated. Julia organizes the different types of meals as a taxonomy and articulates constraints on meal combination. Using a formalism of variables and their values, the meal-planning task involves recalling parts of previous meals followed by adapting and combining the cases using constraint satisfaction. In contrast to the Cadre system, Julia uses logical rather than numerical constraints.

Issues in developing case-based design systems

The application of CBR to design raises a common set of issues regardless of the application domain. These are the broad classifications of representation and process issues we discussed earlier. Representation issues include the content and representation of an individual design-case and case-memory organization. Process issues concern the general process model of a case-based design system—design-case recall and adaptation. Practitioners have addressed these issues in developing and implementing case-based design systems, where each system resolves the issues within the context of its project focus.

Representation of design cases and case memory. Before discussing the representation of design cases, let's look at how cases are considered in CBR. One view is that a case is a story or a lesson to be learned.[4] Other views express cases as information about the context as well as solutions of a problem,[7] or consider that cases record the process by which a problem is solved.[9] Whichever view you take, a case's purpose is to facilitate solving a similar problem in a similar but different context in the future.

A common issue raised in design applications is that a design cannot be told in a single case (or story), or from a single point of view. Design involves reasoning about a design artifact's rationale, structure, behavior, function, constraints among structural elements, decisions, and tradeoffs. A design case's size is an important consideration. Design cases can be divided into smaller chunks and their parts linked as a list, a hierarchy, or a network. The "Complex design cases" section gave examples of different decomposition schemes.

Table 4. What is in a design case?

Approach	System
Design lessons	Archie, Archie-II, AskJef
Function, behavior, and structure variables	Casecad, Cadsyn, Gencad
Formal causal models that explain	Kritik, Cadet
Interpretations as drawing, function, and behavior models	Cadre, Faming
Plan actions and components of physical systems	Panda
Specification units, method to create the solution	Seed
Gestalts, images, vectors, and tuples	Fabel

Table 5. Design-case retrieval methods.

Method	System
User-directed browsing and navigation	Archie, Archie-II, Casecad, AskJef, Janus
Attribute-value matching	Cadsyn, Casecad, Seed, Julia, Fabel, Asp-II, Benton, Panda
Functional indexing	Kritik
Image- and gestalt-based pattern matching	Fabel
Hierarchical problem specifications matching	Seed
Graph matching	Cadet

Case-memory organization refers to the way cases are organized for access during retrieval. Where case memory does not contain many cases, the memory organization is usually a list of pointers to cases. A large case memory requires more efficient access to cases or subcases. AI practitioners commonly use induction to generate a hierarchical indexing tree whose leaf nodes are associated with a list of pointers to cases or subcases. A common issue in organizing design cases is the need to predetermine the features to serve as indices for efficient access to cases. It is difficult, however, to decide beforehand the subset of features that are likely to help in identifying relevance to a new design problem.

Designers resolve the issue of "What is in a design case?" in many ways. Most design stories told by designers or found in design documents do not easily form into cases that can be indexed and classified for reuse. A systematic approach is needed to identify uniform representations and to parse design stories into these formats. The variety of ways for representing designs provides a richer presentation of design information to the user and can directly influence retrieval and adaptation methods.

Design cases can be lessons for helping designers interpret a previous design experience in a similar context, as do Archie, Archie-II, and AskJef. These systems emphasize—both from the point of view of internal representation and in the way they present cases to the user—the results, both good and bad, of key design decisions.

Design cases can classify their variables according to which ones relate to their function, behavior, and structure, so that a case's retrieval and adaptation can focus on one or more of these aspects of a design, as in Casecad and Gencad. These systems emphasize the need for processing different subsets of a design's variables according to the different interests that a user might have at different stages in the design process.

Design cases can consist of models that explain how their structure achieves their functions, as in Kritik and Cadet. In these systems, a case's structure is the solution to a design problem, its function is the design problem that the case solved, and the behavior is an explanation of how the structure achieves its functions. These systems represent and use each of these three types of information differently. In contrast, Casecad and Gencad uniformly represent function, behavior, and structure as attribute-value pairs. Systems using causal models emphasize the ability to use the knowledge embodied in the models to understand a design's internal functioning, and thus to automatically adapt the case.

Design cases can consist of *interpretations*, or a collection of different types of models, as in Cadre and Faming. In these systems, a case links to an interpretation, which consists of three (interrelated) models: a CAD model of the structure of the design, a behavioral model of what the structure does, and a functional model of what is important about what the structure does. As in Kritik and Cadet, these systems emphasize the representation of causal knowledge about designs, but unlike Kritik and Cadet, the representation is done through several interrelated models, represented separately, rather than through one model consisting of several types of information represented together.

Design cases can contain plan actions, components of physical systems, and specification elements, as in Panda and Seed. These systems use the different components of a case representation for different reasoning tasks. For example, they use the specification part of the case during retrieval and the plans during case adaptation.

Another approach represents a case as different views of the same design. For example, in Fabel, design cases can contain images, gestalts, and tuples of values. This project shows that there is no simple response to "What is in a design case?" but rather many answers. The Fabel project also shows that there can be many answers within a single implementation. Representing each case in a variety of ways extends its usefulness to many different situations.

We can consider the representation of design cases according to the way they are stored as computer files, such as text, CAD drawings, and object-oriented models. Many case-based design systems combine representation techniques, as in Cadre, Casecad, Archie, Archie-II, and AskJef (which also has auditory information about cases). These systems emphasize facilitating the presentation of information about a case to the user (that is, information in one or more user-friendly formats), while still permitting the system to operate on the cases.

Design cases can contain a solution generated by a design module, an associated problem specification, and the technology or method used to create the solution, as in Seed. This type of system emphasizes the ability to adapt the procedures used to solve previous problems in new, but similar, contexts, as opposed to only adapting the solution itself.

In summary, the most important issue when defining a case representation is what information in the design facilitates its reuse. Then, the issue of how the CBR tool uses the design-case representation as part of the design process (for retrieval only, for adaptation only, or for both retrieval and adaptation) helps determine the implemented representation of a design case. Table 4 summarizes the approaches of the systems surveyed for this article, using the terminology of the authors of the systems. The approaches are as numerous as there are systems, indicating that this aspect of case-based design has not matured enough to lead to general principles of how to represent a design case.

Indexing and retrieving design cases. In CBR, recalling a case assumes that you know what you are looking for—that is, that the problem is well-defined. Design often involves solving an ill-defined problem in which part of the design process is defining the problem in addition to developing a solution. Designers thus cannot perform the indexing and retrieval involved in design-case recall simply as direct pattern matching. In many implementations, the indexing and retrieval of design cases is done informally, where the user browses through the case library and selects a relevant design case.

The use of informal approaches to recalling relevant design cases addresses the need for flexibility in developing an understanding of the new design problem through browsing and retrieving design cases. The informal approach's effectiveness depends on the number of designs in case memory and the richness of the indexing scheme. If the case library is very large, the informal approach is not very efficient. Linking and cross-indexing the cases allows the user to navigate and browse through the cases in the case library that are related in different ways.

Informal approaches are usually implemented as hypertext browsers with some capability for word search. These systems usually only provide for case browsing and do not help with design-case adaptation. In some systems, such as Cadre, the number of design cases is so small that the user can browse through all stored cases to find the one best for adaptation, and the system assists with the adaptation.

Formal approaches to recalling design cases make assumptions about how a new problem is specified and use the specification for pattern matching. The most common formal method for design-case recall uses a set of attribute-value pairs to specify a new design problem, then matches this set with design cases in memory. A close match is a design case that has many attribute-value pairs in common with the specifications. Many implementations use a *weighted-sum* approach to measure the closeness of the match, where weights define the relative importance of a particular attribute. Although various algorithms are used for pattern matching, many systems use the general approach of matching attribute-value pairs. These include Casecad, Cadsyn, Julia, Fabel, Seed, Asp-II, Panda, and Benton.

Other approaches to finding a relevant case are to index and retrieve based on function, as in Kritik; to retrieve based on matching images or gestalts, as in Fabel; to retrieve based on a hierarchy of problem specifications, as in Seed; and to retrieve using graph-based representations of behavior, as in Cadet. Function retrieval, as in Kritik, uses a single function as the index to each case. This approach describes the new design problem by its intended function and retrieves cases that provide that function. Fabel uses richer pattern-matching methods than the weighted-sum approach; for example, the *fish and sink* algorithm measures the distance between a case and a new problem description. Table 5 summarizes the approaches to case retrieval taken in the systems included for this survey.

A common issue in developing indexing and retrieval schemes for design cases is the need for flexibility and for allowing the design specifications to change or be refined. For example, Kritik analyzes a memory of substances, part of its general domain knowledge, to elaborate on the original design-problem specifications. Casecad and Cadet provide strategies for modifying the new design specification based on the cases retrieved.

APPROACHES ARE AS NUMEROUS AS THERE ARE SYSTEMS, INDICATING THAT THIS ASPECT OF CASE-BASED DESIGN HAS NOT MATURED ENOUGH TO LEAD TO GENERAL PRINCIPLES OF HOW TO REPRESENT A DESIGN CASE.

Another retrieval-related issue is the level of abstraction of cases and the need to decompose a problem to find a relevant design case or subcase. For example, Julia and Cadsyn decompose the problem specification if they cannot find a case to solve the entire problem. The retrieval process then considers a subset of the entire design problem when looking for relevant design cases.

Design-case adaptation. The process of design-case adaptation is essentially the synthesis of a new design solution. The recall of one or more similar design cases provides the starting point for generating a new solution. The adaptation of design cases is the part of case-based design where a new design is generated. CBR applications to design fall into three categories, depending on who or what performs the adaptation:

- the *human designer*,
- a *knowledge-intensive computational method*, or
- a *knowledge-lean computational method*.

Human design-case adaptation. Some systems described here are intended to serve as case libraries—repositories of information about specific designs represented using different formats, with the cases cross-linked to facilitate navigation among related items in the case library. They usually emphasize facilitating case-memory browsing and navigation, and case retrieval. These systems leave what is done with the cases encountered in the memory to the user. Systems taking this approach include Archie, Archie-II, AskJef, Casecad, and Janus. Although these systems do not tackle case adaptation, Archie and Janus do include support for evaluating the user's designs (presumably adapted by the user from previous cases) on the basis of domain models and rules.

Knowledge-intensive case adaptation methods. In many of the applications of CBR to design, automating design-case adaptation relies on constraint satisfaction—for example, in Julia, Cadsyn, Cadre, and Faming (although in Julia it is called constraint posting). Julia handles dynamic constraint variables as well; that is, as the adaptation of a case proceeds, Julia can discover new constraints (or users can specify new ones) that apply to the problem, or it can drop some of the previously considered constraints. Cadsyn uses a predefined set of constraints to evaluate proposed design solutions.

Faming uses numerical constraints to adapt the geometry of a proposed floor plan that is generated by combining cases. Called *dimensional adaptation*, the method consists of dimensionality expansion and reduction. The adaptation strategies in both Faming and Cadre represent the problems they have to solve as networks of constraints; thus, these systems implement case adaptation using algorithms that solve constraint-satisfaction problems. The final solutions provided by constraint-satisfaction algorithms, if any, will not need evaluation because they will be guaranteed to satisfy the constraints.

Table 6. Methods for automated design-case adaptation.

METHOD	SYSTEM
Constraint satisfaction	Cadsyn, Cadre, Faming, Fabel, Julia
Rule-based transformation	Panda, Asp-II, Benton
Subcase combination	Seed, Cadsyn, Panda
Simulation	Asp-II, Kritik, Benton
Causal diagnose and repair	Kritik
Design plan replay	Seed, Asp-II, Benton
Genetic algorithm	Gencad
Generate and test	Fabel

Instead of constraints, other systems employ heuristic modification rules (some of which can implicitly embody constraints) that are specific to the domain, but valid for all artifacts in the domain. Panda, Asp-II, and Benton use a rule-based approach to tweak the parameters in a retrieved case to transform it into a solution to the new problem.

Panda attempts a technique called *reinstantiation* or *subcase replacement* as an alternative for case adaptation. In this method, a submodule from one case adds to, or replaces a submodule in, another case. Panda evaluates a design by comparing the goals it satisfies with the initial user-specified goals. Asp-II uses two alternative, but similar, methods. It uses replanning to adapt a submodule of a case when a rule-based method has successfully adapted the rest of the case. It uses instantiation to adapt a generic description of a design solution by finding appropriate values for its variables when no specific cases that can be adapted have been retrieved from memory. Asp-II evaluates designs both by performing a simulation and by searching for similar cases in memory that represent *failed* designs. Benton employs Asp-II's replanning method for case adaptation to replace fragments of both the design solutions and the design processes stored in its cases. Benton uses syntactic and other knowledge related to its domain to verify the correctness of its designs.

Another case-adaptation approach combines model-based reasoning with CBR. The cases provide the starting point of the solution to a given problem. Models provide knowledge to fix the areas where cases do not quite satisfy the current problem specification. Kritik considers case adaptation to be a diagnosis and repair problem, and implements case evaluation as a simulation process. Case-specific models include behaviors, which explain how a design's structure achieves its function. Kritik analyzes (diagnoses) these models and identifies candidate modifications (repairs). Case adaptation is plan-based—the system knows and can attempt different methods or plans for adaptation. Generic adaptation methods in Kritik include component replacement, component replication, and substance replacement. After performing a repair, Kritik simulates the resulting model to determine whether the resulting structure achieves the intended functions, and thus whether adaptation worked.

Fabel offers a variety of adaptation methods that depend on the aspect or view of the case being adapted, and uses techniques such as generate-and-test and constraint satisfaction. Thus, as in many of the other systems we've discussed, Fabel adopts a hybrid framework combining several methods.

An alternative view of case adaptation lets the user interact with the system, thus combining the automation of tedious, repetitive, formalizable tasks, with the intelligent decision-making capabilities of users in important stages of the design. For example, Seed can autonomously perform several case-adaptation operations, but the user must select which ones to attempt at which times. The case-adaptation operations available to the user in Seed implement the addition, removal, translation, modification, aggregation, and disaggregation of design components, and the alteration of their attributes. Seed supports the evaluation of designs with respect to user-specified design requirements.

Knowledge-lean case adaptation. Case adaptation and evaluation in the systems we've described rely on access to generic domain knowledge such as constraints, rules, plans, or models. However, systems can also perform case adaptation without relying on such domain knowledge. Our current work on Gencad focuses on design-case adaptation. Within the subtasks of case adaptation, we employ a genetic algorithm to eliminate the need for expertise related to both the proposal and the modification of potential solutions to design problems.

Gencad contains a case library of 61 structural engineering designs of tall buildings. It retrieves all partially matching cases when a new design problem is specified. The retrieved cases serve as the initial population of Gencad's genetic algorithm. The genetic algorithm uses both crossover and mutation to generate variations on the population's cases, each of which is a potential solution to the design problem. Domain knowledge in the form of constraints, together with the user's design-problem specifications, assist the evaluation of potential solutions. If none of them is good enough, Gencad selects the best of the potential solutions generated and left over from previous generations of the genetic algorithm to participate in a new round of crossover, mutation, and evaluation, and the process repeats.

In further research on Gencad, we're trying to eliminate the need for large amounts of knowledge for evaluating potential design solutions, as well as for their generation and modification. We will do this by training a neural network to recognize good solutions (using the 61 cases in Gencad's memory as examples of good solutions) and using it to evaluate the solutions resulting from the genetic algorithm. The neural network will eliminate the need to explicitly code either constraints or heuristic rules about the domain of structural engineering that are currently necessary to evaluate the results produced by the genetic algorithm.

Summary of automated design-case adaptation methods. The choice of case adaptation methods used by the different systems depends largely on what kind of knowledge is available about a domain, as well as the form in which the knowledge, and the design cases themselves, is available or represented. A general model for design-case adaptation is to propose a design solution, evaluate the proposed design solution, and modify the design until evaluation is satisfactory. However, this general description does not provide any insight into how to implement design-case adaptation. Table 6 summarizes the systems described just now.

THE USE OF CBR IN DESIGN HAS grown as a rich source of inspiration for how to apply the CBR paradigm to a new design application. The variety and complexity of design domains that have demonstrated the use of CBR have addressed many issues that the paradigm itself does not directly address. The most consuming effort in developing a case-based design system is the development of the case memory. This is equivalent to developing the knowledge base or grammar for the knowledge-based and language approaches to modeling design.

On the other hand, the development of

Coming in June

QUALITATIVE REASONING

Qualitative understanding of behavior and the capability to reason with qualitative knowledge remains crucial in both human and machine intelligence despite—or some would say because—more and more sophisticated techniques are used in the modeling and analysis of everything from electro-mechanical devices to such nonphysical systems as the stock market. Guest edited by Yumi Iwasaki of Stanford's Knowledge Systems Lab, this special issue will document recent progress in this area by looking at

- **Modeling and simulation of complex industrial processes**
- **Model-based automatic generation of sequence control programs from design information**
- **Gas turbine condition monitoring using qualitative model-based diagnosis**
- **Using qualitative physics to create articulate software for science and engineering education**
- **Qualitative simulation and prediction**
- **Qualitative models as indices for memory-based prediction**
- **NASA's current work on autonomous spacecraft**

More AI News You Can Use

case-based design systems has inspired much research on AI in design. The use of CBR as an umbrella for understanding how to apply formal models of design knowledge or experience to a new design problem has become common. One example of this hybrid approach is the combination of case- and model-based reasoning. The concepts in CBR for memory organization, indexing, and retrieval can apply directly to model-based reasoning. We see the distinction blur in systems such as Kritik. The generality of the CBR paradigm shows that it can be applied to many AI approaches to design.

Finally, the issue of design assistance or design automation has led to two types of case-based design systems: those that provide a design memory with facilities for indexing and retrieving previous designs, and those that automatically retrieve or adapt previous designs. The distinction refers to different expectations about the role of the user, or human designer. This distinction is not a major issue in our presentation of case-based design systems. The issues raised by applying CBR to design center around the representation of design cases. The development of a design-case memory is informed by AI techniques and design research. The use of the design memory for design support or design automation might influence the user-interface design and the development of algorithms for changing design cases, but does not change the need to develop formal representations of design experiences.

References

1. R.C. Schank, *Dynamic Memory: A Theory of Learning in Computers and People*, Cambridge Univ. Press, Cambridge, UK, 1982.
2. J.L. Kolodner and C.K. Riesbeck, *Experience, Memory, and Reasoning*, Lawrence Erlbaum Associates, Hillsdale, N.J., 1986.
3. C. Riesbeck and R.C. Schank, *Inside Case-Based Reasoning*, Lawrence Erlbaum, 1989.
4. J.L. Kolodner, *Case-Based Reasoning*, Morgan Kaufmann, San Francisco, 1993.
5. P. Jackson, *Introduction to Expert Systems*, 2nd ed., Addison-Wesley, Reading, Mass., 1990.
6. P. Pu, "Special Issue: Case-Based Design Systems," *Artificial Intelligence in Engineering, Design and Manufacturing*, Cambridge Univ. Press, 1990.
7. M.L. Maher, B. Balachandran, and D.M. Zhang, *Case-Based Reasoning in Design*, Lawrence Erlbaum, 1995.
8. M.L. Maher and P. Pu, eds., *Issues and Applications of Case-Based Reasoning in Design*, Lawrence Erlbaum, 1997.
9. U. Flemming et al., "Case-Based Design in a Software Environment That Supports the Early Phases," in *Issues and Applications of Case-Based Reasoning in Design*, M.L. Maher and P. Pu, eds., Lawrence Erlbaum, 1997.

Mary Lou Maher is codirector of the Key Centre of Design Computing and an associate professor in the Department of Architectural and Design Science at the University of Sydney, Australia. Her research interests include case-based reasoning in design, modeling creative design using genetic algorithms, and computer support for collaborative design. She is the editor or coauthor of eight books and over 100 papers on design computing. She received her BS in civil engineering at Columbia University and her MS and PhD in civil engineering at Carnegie Mellon University. She can be contacted at the Dept. of Architectural and Design Science, Univ. of Sydney, NSW 2006, Australia; mary@arch.su.edu.au; http://www.arch.su.edu.au/~mary/.

Andrés Gómez de Silva Garza is a PhD student in the Department of Architectural and Design Science at the University of Sydney. His research interests include case-based reasoning, genetic algorithms, and design science. He received his BSc in computer engineering from the National Autonomous University of Mexico in 1991, and his MSc in computer science from the Georgia Institute of Technology in 1993. He is a member of the AAAI. He can be contacted at the Dept. of Architectural and Design Science, Univ. of Sydney, NSW 2006, Australia; andres@arch.su.edu.au; http://www.arch.su.edu.au/~andres/index.html.